

Presentation 1

What did each of the team members do?

Person 1: Henry -- Understanding the code organization focusing on data per-processing step, analyzing code for errors or unused code, and executing code and interpreting outputs.

Person 2 & 3: Zijin & Vicky -- model training and testing

Vicky: Understand the codes Comment whether this is the most efficient code, or are there redundant code blocks / operations

Zijin: Execute the code, produce, and analyze results.

Person 4: Abhi -- Understanding the mathematical operations behind the steps in the code, from data preprocessing to backpropagation. Covers tensor transformations, attention calculations, gradient updates, etc.

Person 5: Eunice (dropped the course) -- data visualization and analysis (need someone to take this role)

How does it help the project?

Zijin: analyze past model's results and identify directions for improvement. The model shows strong overall performance (test AUC 0.875), and the errors are false alarms. Additional analysis suggests these failures are not simply explained by patch count or by evidence concentration alone. From this, I propose plans for next steps (will be discussed in the following sections)

Vicky: By reviewing the code for data pre-processing, the CNN model, and the attention mechanism, I have gained a better understanding of the overall workflow, which is essential for my future programming needs. Identifying code efficiencies and redundancies helps me iterate on the model more effectively and improve its execution speed.

Henry: Understanding the code structure allows me to identify parts where it is unclear what is being done so that in the future those sections might be made more intuitive. It will also allow me to recognize the sections relevant to modify if we are to implement some of the suggestions in the project spreadsheet and how it is interacting with other parts of the data prediction pipeline.

Abhi: Understanding the math behind models will add an element of interpretability to deep neural networks. It is also helpful for informing architectural and training choices by understanding what happens to the data behind the scenes. Debugging with shape incompatibilities will have more clarity by seeing how tensors are transformed through different layers.

Issues faced that haven't been resolved:

- Task Split: Need to split the tasks again because someone in our group dropped the course.
- Model Training Level: Check the cases with the biggest gaps, like case 75, 44, 46, and 23.
- Code interactivity: Inability to run only selected code chunks reduces understanding of intermediary steps. Could maybe look into converting the .py files back into jupyter notebooks in order to run the code chunks individually.
- Patches: Lack of visualization on how the StainBagCaseDataset randomly selects patches and transforms them. Is it picking a lot of small uninformative patches because there are more of them? Is it cropping out the epithelium boundary some times and not others? Does the absolute scale of the patches affect CNN filter matches?

Proposed next steps:

Model Training and Testing (Zijin): audit high-gap cases, refine preprocessing to keep informative tissue patches and remove low-information ones, add the class-balance check and use a class-weighted loss, and prioritize borderline benign cases during training to reduce benign false positives.

Also, I am still learning the CNN model since I haven't taken STAT 362.

Model Training and Testing (Vicky): optimize redundant code for faster performance, learn CNN fundamentals (since I haven't taken STAT 362), and begin scaling the model by incorporating more cases.

Data-preprocessing (Henry): Continue to try to understand the tensor transformations and look into removing RandomResizeCrop from transforming the training data because it might be cutting off important contextual information from the patch images.

Mathematical Operations (Abhi): Work out the math behind different model architectures and training strategies to assess effectiveness. Continue to understand tensor transformations to help with future debugging.

References

- Lecture: STAT 390: Data science project CMIL Classification Problem statement & Data
- Visualization_Prediction — Fall 2025 MIL Trainer Final Model and Analysis
- 2025 Fall Presentation Slides
- Jack Yu's presentation going over the outline of the code from last quarter

Links to the presentation & code:

- Presentation: [Presentation1_Team6](#)
- Code: [Code1_ModelTraining_29Jan](#)